

Rapport final - Vulnerabilites et recommandations

Stagiaire: Mohamed Habib Baili

Encadrant: Baylassen

Entreprise: L2T

Projet: Krayin Laravel CRM

Date: 2026-06-28

Rapport final - Vulnérabilités, impacts et recommandations

Identification

- Stagiaire: Mohamed Habib Baili
- Encadrant: Baylassen
- Entreprise: L2T
- Projet: Mise en place d'une pipeline DevSecOps pour Krayin Laravel CRM
- Date: 2026-06-28

1. Résumé exécutif

Le projet Krayin CRM a été préparé pour intégrer une pipeline DevSecOps automatisée. Cette pipeline couvre la récupération du code, le build, les tests, l'analyse qualité SonarQube, les audits de dépendances, l'analyse statique de sécurité, la recherche de secrets, un scan filesystem et un scan dynamique OWASP ZAP.

Les tests applicatifs existants passent correctement:

```
5 tests passés, 15 assertions
```

Le build frontend passe également:

```
vite build: succès
```

Les audits de dépendances ont toutefois identifié des vulnérabilités à corriger.

2. Vulnérabilités identifiées

VULN-001 - Dépendance PHPSpreadsheet vulnérable

- Package: `phpoffice/phpspreadsheet`
- Sévérité: Critique
- Type de risque: traitement de fichiers bureautiques potentiellement dangereux
- Impact: selon le contexte d'utilisation, un fichier malveillant pourrait provoquer une exploitation ou un comportement non prévu.
- Recommandation: mettre à jour le package via Composer avec ses dépendances.

VULN-002 - Laravel Framework vulnérable

- Package: `laravel/framework`
- Sévérité: Haute/Moyenne selon advisory
- Risques: confusion de chemins sur URL signées, injection CRLF dans certaines règles email.
- Impact: contournements applicatifs possibles si les fonctionnalités affectées sont utilisées.
- Recommandation: mettre à jour Laravel vers une version corrigée compatible avec le projet.

VULN-003 - Symfony Mailer/Mime vulnérable

- Packages: `symfony/mailer`, `symfony/mime`
- Sévérité: Haute/Moyenne
- Risques: injection d'en-têtes email ou injection de commande SMTP dans certains cas.
- Impact: risque important si l'application envoie des emails avec des entrées utilisateur non contrôlées.
- Recommandation: mettre à jour les composants Symfony et valider strictement les adresses email.

VULN-004 - Guzzle et PSR-7 vulnérables

- Packages: `guzzlehttp/guzzle`, `guzzlehttp/psr7`
- Sévérité: Moyenne
- Risques: confusion de domaines, CRLF injection, downgrade proxy.

- Impact: risque sur les appels HTTP sortants, surtout si des URLs proviennent d'entrées utilisateur.
- Recommandation: mettre à jour Guzzle et PSR-7, limiter les domaines externes autorisés.

VULN-005 - Vite et esbuild vulnérables

- Packages: vite, esbuild
- Sévérité: Haute/Moyenne
- Risques: path traversal, exposition de fichiers via serveur de développement.
- Impact: surtout critique si le serveur Vite dev est exposé sur un réseau non fiable.
- Recommandation: ne jamais exposer Vite en production, mettre à jour Vite après test de compatibilité.

3. Impact global

Domaine	Niveau de risque	Commentaire
Dépendances PHP	Élevé	Présence d'une vulnérabilité critique et de plusieurs hautes/moyennes
Dépendances JS	Moyen	Risque surtout lié au serveur de développement
Tests automatisés	Faible à moyen	Tests existants OK mais couverture limitée
Qualité code	À confirmer	SonarQube doit être exécuté en CI avec token
DAST	À confirmer	ZAP est configuré mais les scénarios authentifiés restent à enrichir

4. Plan de correction priorisé

Priorité 1 - Corriger les dépendances critiques et hautes

```
composer update phpoffice/phpspreadsheet laravel/framework symfony/mime symfony/mailler
--with-all-dependencies
npm update vite
```

Puis relancer:

```
composer audit
npm audit
php artisan test --compact
npm run build
```

Priorité 2 - Renforcer la configuration de sécurité Laravel

- APP_DEBUG=false hors local.
- HTTPS obligatoire en production.
- Cookies de session avec secure, httponly, samesite.
- Rotation régulière des secrets.
- Pas de fichier .env dans Git.

Priorité 3 - Étendre les tests

Ajouter des tests pour:

- Création/modification/suppression de leads.
- Permissions utilisateurs et rôles.
- Accès interdit aux ressources non autorisées.
- Validation des entrées email.
- Upload/import de fichiers.

Priorité 4 - Compléter les tests DAST

Créer des scénarios ZAP authentifiés pour:

- Broken authentication.
- Session hijacking.
- Privilege escalation.
- SQL injection dans formulaires.
- API abuse.

5. Conclusion

La pipeline DevSecOps demandée est préparée et couvre les étapes majeures du cahier des charges. Les tests et le build passent localement. Les analyses de dépendances ont révélé des vulnérabilités réelles qui doivent être corrigées avant une mise en production.

La prochaine étape recommandée est de pousser cette configuration sur GitHub, configurer SonarQube, exécuter la pipeline complète, puis mettre à jour les dépendances vulnérables.